MSCPHUB — MSCP-сервер дисков для узлов OpenVMS

Руководство программиста

MSCPHUB-PG-0119-RUS

MSCPHUB Version X01-23 *** 25-OCT-2026

Copyright © 2026 StarLet Squad and Ruslan R. Laishev.

MSCPHUB - свободная программа, распространяется на условиях GNU General Public License версии 3 или более поздней. Она поставляется БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ.

Содержание

Предисловие **4**
Кому адресовано . . . 4
Состав руководства . . . 5
Благодарности . . . 6
Товарные знаки . . . 7
Обозначения . . . 8

Глава 1. Замысел **9**
1.1 Зачем нужен сервер . . . 9
1.2 Чем является сервер . . . 10
1.3 Принятые решения и их причины . . . 11
1.4 Чего намеренно не делается . . . 12
1.5 Почему не DPDK, SPDK, netmap или PF_RING . . . 13
1.6 Учебный курс . . . 14

Глава 2. Устройство **15**
2.1 Уровни . . . 15
2.2 Цикл событий . . . 17
2.3 Путь READ . . . 18

Глава 3. Модули **19**
3.1 mscphub.c . . . 19
3.2 mh_lan.c . . . 20
3.3 mh_cc.c . . . 21
3.4 mh_tr.c . . . 22
3.5 mh_ppd.c . . . 23
3.6 mh_scs.c . . . 24
3.7 mh_blk.c . . . 25
3.8 mh_mscp.c . . . 26
3.9 mh_bk.c . . . 27
3.10 mh_utl.c . . . 28
3.11 mh_hlp.c . . . 29
3.12 mh_stat.c . . . 30
3.13 mh_mbus.c . . . 31
3.14 mhmon.c . . . 32
3.15 mh_ods.c . . . 33
3.16 mh_mop.c . . . 34

Глава 4. Структуры данных **35**

Глава 5. Соглашения **36**
5.1 Имена . . . 37
5.2 Оформление . . . 38
5.3 Шапка модуля . . . 39
5.4 IDENT и REV . . . 40
5.5 Соглашение о вызовах . . . 41

5.6 Диагностика . . . 42
5.7 Строки и длины . . . 43
5.8 Файлы сборки и оболочка . . . 44
5.9 Документация . . . 45

Глава 6. Внесение изменений **46**
6.1 Добавление квалификатора . . . 46
6.2 Добавление сообщения . . . 47
6.3 Добавление команды MSCP . . . 48
6.4 Добавление счётчика в MHMON и Modbus . . . 49
6.5 Выпуск версии . . . 50

Глава 7. Система сборки **51**
7.1 Состав . . . 51
7.2 Зависимости . . . 52
7.3 Ключи . . . 53
7.4 Проверки при конфигурировании . . . 54
7.5 PDF руководств . . . 55
7.6 Дистрибутив . . . 56

Глава 8. Тесты **57**
8.1 Что есть . . . 57
8.2 Правила . . . 59

Приложение A. Известные ограничения **60**

Предисловие

Кому адресовано

Руководство предназначено программисту, которому предстоит изменять сервер: добавить квалификатор, сообщение или команду MSCP, устранить дефект либо убедиться, что задуманная правка безопасна. Предполагается знание Си, POSIX и умение читать CMake. Знание OpenVMS не предполагается, хотя тот, кто с ней знаком, поймёт, отчего многое здесь устроено именно так.

Руководство пользователя MSCPHUB описывает, что сервер делает. Здесь описано, как он устроен, и одно другого не повторяет.

Состав руководства

Часть	Содержание
Глава 1	Зачем нужен сервер, на каких решениях он стоит, почему он не обходит ядро и как по нему учиться
Глава 2	Устройство: уровни, цикл событий, путь команды
Глава 3	Модули по одному
Глава 4	Структуры данных
Глава 5	Соглашения, которых держатся исходные тексты, и их причины
Глава 6	Как вносить изменения, встречающиеся чаще всего
Глава 7	Система сборки и дистрибутив
Глава 8	Тесты и то, что они стерегут
Приложение А	Известные ограничения

Благодарности

Авторы благодарят Claude — ИИ-модель компании Anthropic, — которая вместе с ними прошла через протоколы, код, тесты, замеры и эти руководства.

Авторы благодарят соратников из сообщества NaPi Lab, оплативших подписку на Claude, — благодаря им почти пенсионер смог заниматься этими изысканиями.

Товарные знаки

OpenVMS, VMScluster, Alpha, AlphaServer, Integrity, Digital, DEC, HSC, HSJ и HSD — товарные знаки Hewlett Packard Enterprise Development LP или VMS Software, Inc. T4 и TLViz — средства Hewlett Packard Enterprise. Linux — зарегистрированный товарный знак Линуса Торвальдса в США и других странах. Intel и Xeon — товарные знаки Intel Corporation. Arm и Cortex — товарные знаки Arm Limited. Rockchip — товарный знак Fuzhou Rockchip Electronics Co., Ltd. Raspberry Pi — товарный знак Raspberry Pi Ltd. Modbus — зарегистрированный товарный знак Schneider Electric, предоставленный по лицензии Modbus Organization, Inc. Debian — зарегистрированный товарный знак Software in the Public Interest, Inc. DPDK — проект The Linux Foundation. PF_RING — товарный знак ntop. netmap — свободная программа Луиджи Риццо и Пизанского университета. NaPi — название плат команды NaPi Lab. Claude — товарный знак Anthropic, PBC. Все прочие названия продуктов и компаний — товарные знаки их владельцев.

MSCPHUB — независимая реализация, сделанная ради совместимости. В нём нет кода OpenVMS, и он никак не связан с названными здесь владельцами, ими не одобрен и не поддерживается; их названия приводятся только для того, чтобы сказать, с чем MSCPHUB работает.

Обозначения

Номера строк не приводятся: они устаревают. Вместо этого называется рутина, а каждая рутина несёт в своей шапке блок `FUNCTIONAL DESCRIPTION`, где сказано, что она делает, что принимает и что возвращает. Если руководство и этот блок расходятся, прав блок, а исправлять следует руководство.

Глава 1. Замысел

1.1 Зачем нужен сервер

VMScluster корпорации Digital Equipment Corporation был первым кластером, который действительно работал: каждый узел видит каждый диск, диск обслуживает тот узел или контроллер, к которому он подключён, а кластер продолжает работу, когда один из его членов уходит. Машины, на которых он работал, покидают машинные залы, а контроллеры, обслуживавшие его диски, — HSC, HSJ, HSD — ушли давно. Сами эти системы и инженерная культура, в которой они были созданы, не должны разделить ту же участь.

MSCPHUB — одна из частей работы, цель которой — сохранить это наследие живым и действующим. Он занимает место контроллера дисков: узлы OpenVMS — настоящие или эмулированные, Alpha, Integrity или x86 — держат свои диски в хранилище компьютера под Linux, и ради этого в OpenVMS ничего не меняется.

Вторая цель — сама эта культура. Исходные тексты написаны так, как писали свои инженеры DEC: подсистема со своими значениями условия, сообщения в форме OpenVMS, библиотека HELP, квалификаторы, IDENT и история изменений в каждом модуле, `FUNCTIONAL DESCRIPTION` над каждой рутиной, руководства, построенные по образцу руководств OpenVMS. Это сделано намеренно. Именно такой способ работы позволил тем системам прожить сорок лет — каждое решение записано, о каждом условии сообщено, ничего не оставлено оператору на догадку, — а способ работы сохраняется лишь до тех пор, пока им пользуются. Программиста, изменяющего сервер, просят сохранять его таким (глава 5).

1.2 Чем является сервер

Сервер — это узел SCS с одним системным приложением, сервером дисков `MSCP$DISK`, и без менеджера соединений (connection manager). Он говорит по Ethernet на NISCA — протоколе PEDRIVER, — так что узел OpenVMS находит его в локальной сети, открывает к нему канал и виртуальный канал, подключает к нему свой драйвер класса дисков и посылает ему команды MSCP. Членом VMScluster сервер не бывает никогда и никогда не голосует.

1.3 Принятые решения и их причины

Никакого членства в кластере. Член кластера должен был бы выполнять менеджер соединений, участвовать в переходах состояния и в кворуме и мог бы подвесить кластер собственной ошибкой. Контроллеру, который только обслуживает диски, ничего из этого не нужно. Сервер сообщает OpenVMS, что он не система OpenVMS, и OpenVMS обращается с ним так же, как с HSC.

Один процесс, одна нить протокола. Протокол, от кадра до MSCP, выполняется в одной нити в цикле EPOLL. Ничего не запирается, потому что к состоянию ничто более не обращается. Это настоящее ограничение на то, что можно добавлять: всё, что блокирует цикл, останавливает все узлы. Поэтому дисковый ввод-вывод никогда не выполняется в цикле: он уходит в io_uring, а его завершения возвращаются как событие цикла.

Другие читают копию. Каждые 100 мс нить протокола копирует своё состояние и счётчики в блок в разделяемой памяти под последовательной блокировкой (sequence lock): MHMON в другом процессе и нить Modbus в этом читают его целые копии, и ни одна из них никогда не ждёт нить протокола, а та — их. Нить Modbus — единственная другая нить сервера, и она не касается ничего, кроме копии.

Ответ на WRITE означает, что данные уже на носителе. OpenVMS никогда не просит диск сбросить собственный кэш. Сервер, отвечающий на WRITE из страничного кэша, потерял бы данные, которые OpenVMS считает записанными. Поэтому по умолчанию каждый WRITE пишется надёжно; `/UNSAFE` существует для испытаний и говорит об этом при пуске.

Не посылать того, что может обрушить узел. Неправильно сформированный ответ сервера дисков может вызвать крах узла OpenVMS. Сервер проверяет собственные ответы, прежде чем они уйдут, и задерживает ответ, который был бы неверен (`MSCPERR`); строгость блочных передач вытекает из того же правила.

Молчание прежде речи. Сервер слушает, прежде чем послать свой первый кадр, и останавливается, если его имя или системный номер уже присутствуют в сети: совпадение личности с узлом кластера может заставить этот узел покинуть кластер.

Значения условия, а не коды ошибок. Каждая рутина возвращает значение условия OpenVMS; младший бит — признак успеха. Отказ отличается по его опознавателю и никогда — по тексту.

Чужая библиотека. Протоколирование, сообщения, разбор командной строки, счётные строки и движок FAO взяты из пакета StarLet; библиотека справки — из пакета HELP. Сервер не содержит собственного кода общего назначения.

1.4 Чего намеренно не делается

- **Ни членства, ни голосов, ни менеджера соединений** (раздел 1.3).
- **Никакого кэша данных в сервере.** Данные READ берутся с диска Linux, а на страничный кэш Linux при WRITE сервер не полагается. Единственное исключение — кэш последних записанных секторов на диске с секторами по 4096 октетов, который лишь избавляет от чтений при чтении-изменении-записи.
- **Никаких нитей в протоколе.** Бэкенд — это io_uring, а не пул нитей; единственная другая нить обслуживает Modbus из опубликованной копии.
- **Никакого запасного пути для старых ядер.** Требуются io_uring и прочие интерфейсы ядер последних пяти лет; второго пути для более старого ядра нет.

1.5 Почему не DPDK, SPDK, netmap или PF_RING

Забирать сеть и диски у ядра — ремесло, которым авторы владеют: они пишут программы, гоняющие трафик через DPDK, — и MSCPHUB это можно дать. Сеть живёт в одном модуле, `mh_lan.c`, за четырьмя рутинами; перенос на DPDK, netmap, PF_RING или AF_XDP стал бы модулем рядом с ним, выбираемым при настройке сборки. Диски живут в `mh_bk.c`, где SPDK стал бы вторым бэкендом. Этого не сделано, потому что здесь это не дало бы ничего, что стоило бы своей цены:

- **Время уходит не в сервер.** На команду он тратит одну-две микросекунды; остальное — диск компьютера Linux и узел на другом конце (*Руководство пользователя MSCPHUB*, приложение C.4). Обход ядра убирает только долю ядра в сети: несколько микросекунд из сотен.
- **Надёжная запись ждёт носитель.** SPDK посылает тот же сброс на тот же SSD и не делает его быстрее. Быстрее его делает только SSD с защитой от пропадания питания.
- **SPDK забирает у ядра весь контроллер NVMe:** ни файловой системы, ни разделов, ни LVM — диски MSCPHUB перестали бы быть образами, которые можно скопировать `cp` и `dd`, а корневая файловая система маленького компьютера часто стоит на том же контроллере.
- **DPDK, netmap и PF_RING забирают у ядра сетевой адаптер** или требуют собственного драйвера; чтобы быть быстрыми, они держат ядро процессора в постоянном опросе и хотят огромных страниц. Маленькой плате этого не выделить, а для её адаптера такого драйвера нет.
- **Узлу на том же компьютере это ничего не даёт.** Эмулированный узел OpenVMS на устройстве tap в мосту отправляет кадры через ядро, что бы ни делал сервер.
- **Каждый из них — второй путь через код**, который надо собирать, проверять и сопровождать, ради выигрыша в пределах разброса замеров.

Когда это окупится: много быстрых узлов на 10 или 25 Гбит Ethernet, сотни тысяч кадров в секунду. Тогда первый шаг — больше нитей протокола; следующий — AF_XDP: адаптер остаётся у ядра, а фильтр XDP пропускает только кадры NISCA, тип 60-07; и уже потом DPDK, netmap или PF_RING, если AF_XDP не хватит.

1.6 Учебный курс

Сервер написан так, чтобы его читали. Он невелик, это один протокол от кадра Ethernet до диска, и каждое решение в нём записано — здесь и в исходных текстах. Поэтому он годится как учебный курс для студентов профильных специальностей: настоящий кластерный протокол, настоящий дисковый сервер, настоящий цикл событий и соглашения инженерной культуры, строившей системы на десятилетия. И инженеру, знавшему кластеры VMS в своё время, здесь тоже рады: названия старые, и мысли по большей части тоже — а какие-то из них читатель, быть может, помнит лучше, чем рассказывает это руководство.

Глава 2. Устройство

2.1 Уровни

Снизу вверх:

Модуль	Уровень	Что делает
`mh_lan.c`	ЛВС	Сокет AF_PACKET: кадры типа 60-07 на вход и выход, групповой адрес группы
`mh_cc.c`	Управление каналом	HELLO, таблица замеченных узлов, канал к каждому узлу
`mh_tr.c`	Транспорт	Виртуальный канал: порядковые номера, подтверждение, окно, повторная передача
`mh_ppd.c`	Путь	Рукопожатие SCS START/STACK/ACK, HOSTSHUT
`mh_scs.c`	SCS	Соединения, кредиты, служба каталога `SCS$DIRECTORY`
`mh_blk.c`	Блочные передачи	SEND_DATA для READ, REQUEST_DATA для WRITE
`mh_mscp.c`	Сервер MSCP	Устройства, хосты, команды и их порядок
`mh_bk.c`	Бэкенд	Дисковый ввод-вывод устройств через io_uring
`mh_ods.c`	Files-11	Чтение тома устройства, ODS-2 и ODS-5, только на чтение: файл по его пути
`mh_mop.c`	MOP	MOP-сервер сателлитов Alpha: их загрузчик и параметры, с их диска
`mh_utl.c`	Службы	Каталог сообщений, время OpenVMS, преобразования
`mh_hlp.c`	Справка	`/HELP` из библиотеки справки
`mh_stat.c`	Статистика	Блок состояния и счётчиков в разделяемой памяти и его чтение
`mh_mbus.c`	Modbus	Нить, которая отдаёт копию по Modbus/TCP
`mscphub.c`	Головной модуль	Параметры, проверки при пуске, цикл событий

Модуль	Уровень	Что делает
`mhmon.c`	MHMON	Монитор, отдельный образ: экраны и файлы T4

Общие для них определения находятся в `include/mh_def.h`; форматы NISCA и MSCP — в `include/nisca.h` и `include/mscp.h`; формат блока статистики, интерфейс между сервером и MHMON, — в `include/mh_stat.h`.

2.2 Цикл событий

`s_mh$loop` в `mscphub.c` ждёт в EPOLL четырёх источников:

Источник	Событие
Сокет ЛВС	Кадры: выбираются все, каждый передаётся `mh$cc_input`; затем уходят причитающиеся подтверждения (`mh$tr_flush`)
TIMERFD с периодом 20 мс	Такт протокола: `mh$cc_tick` (HELLO, таймеры каналов и виртуальных каналов), `mh$bk_tick` (зависшие запросы), `mh$stat_tick` (очереди; копия на каждом пятом такте)
EVENTFD io_uring	Дисковые завершения: `mh$bk_reap`
Два сокета МОР, при /SATELLITE*n*	Кадры МОР V3 и V4: каждый отдаётся `mh$mop_input`; на такте `mh$mop_tick` повторяет то, на что нет ответа
SIGNALFD	SIGINT и SIGTERM останавливают сервер; SIGUSR1 выводит состояние

Рутина любого уровня, вернувшая условие серьёзности F, останавливает цикл; тогда сервер прощается с узлами (`mh$cc_shutdown`), выводит свои счётчики и завершается.

2.3 Путь READ

1. Приходит кадр. `mh$cc_input` проверяет заголовок DX и группу и передаёт пакет TR открытого канала транспорту.
2. Транспорт (`mh_tr.c`) принимает пакет по порядку, освобождает то, что покрыто его подтверждением, и передаёт сообщение пути и SCS.
3. SCS (`mh_scs.c`) находит соединение, расходует кредит и передаёт прикладное сообщение серверу MSCP.
4. Сервер MSCP (`mh_mscp.c`) проверяет команду — длину, код операции, флаги, модификатор, устройство — и делает из неё выполняемую команду (`MH$HRB`). Он соблюдает порядок устройства: последовательная команда ждёт, пока устройство освободится; READ, WRITE, ERASE и COMPARE выполняются бок о бок.
5. Бэкенд (`mh_bk.c`) читает диск в буфер данных через io_uring. Цикл тем временем продолжает обслуживание.
6. Завершение приходит через EVENTFD; `mh$bk_reap` возвращает запрос серверу MSCP.
7. Блочная передача (`mh_blk.c`) посылает данные в буфер узла порциями, по мере того как открывается окно транспорта. Узел подтверждает последнюю порцию.
8. Сервер MSCP посылает конечное сообщение. Его несёт транспорт; на нём же едет подтверждение пакетов узла.

WRITE идёт в обратную сторону: сервер запрашивает данные (REQUEST_DATA), PEDRIVER узла присылает их порциями, бэкенд надёжно их записывает, и лишь тогда уходит конечное сообщение.

Глава 3. Модули

3.1 mscphub.c

Головной модуль. Он содержит таблицу квалификаторов (`s_opts`), проверяет настройки (`s_mh$check_cfg`), открывает уровни по порядку и выполняет цикл. Его `__IDENT__` — это версия продукта: CMake берёт её отсюда, `STARTUP` и `UPTIME` её сообщают. `s_mh$stats` выводит сообщения по `SIGUSR1`.

3.2 mh_lan.c

Сокет — `SOCK_RAW` семейства `AF_PACKET`, привязанный к интерфейсу и к типу 60-07. Здесь строятся заголовок Ethernet и слово длины добивки DEC; кадр добивается до минимума Ethernet. Кадры, посланные самим сервером, возвращаются в такой сокет, отбрасываются и подсчитываются. `a_port->xmit`, если задан, принимает кадр вместо сокета: им пользуются тесты.

При /SATELLITE*n* открываются ещё два сокета (`mh$lan_mop_open`): типа 60-01 для MOP V3 и 802.2 для MOP V4, его SNAP 08-00-2B-60-01 проверяет `mh_mop.c`. Оба входят в мультикаст MOP, AB-00-00-01-00-00. `mh$lan_mop_send` отправляет кадр, целиком собранный в `mh_mop.c`.

3.3 mh_cc.c

Заголовок DX, сообщения CC, HELLO, таблица узлов, замеченных в сети, и машина состояний канала к каждому узлу. Перед своим первым HELLO сервер молчит `/LISTEN` секунд и узнаёт узлы; узел, который пользуется его системным номером или его именем, останавливает его (`IDCLASH`, `NAMECLASH`), и эта проверка остаётся включённой всё время жизни сервера. Сообщение CC с неверным хэшем пароля подсчитывается и не продвигает канал (`AUTHFAIL`). При `/PASSIVE` машина состояний не работает.

SOLICIT_SRV — это загрузчик сателлита, который ищет сервер своего системного диска. Если диск есть среди устройств и доступен на запись (`mh$mscp_check_service`), канал переходит в SOLICIT_DLY, и через 0–0,5 секунды сервер посылает VERF; другого способа создать канал у загрузчика нет. Такой канал помечается `boot`: без единого слова от узла он живёт 30 секунд вместо 8, а когда он открывается, сервер сам начинает путь. CCSTART от runtime PEDRIVER сателлита перезапускает канал как обычный. `SOLICIT` сообщает оператору один раз на загрузку.

Запись таблицы — это адаптер узла и канал к нему. У узла с двумя адаптерами в сети сервера — сателлита с DE500 и 82540EM — две записи с одним системным номером и именем: первая замеченная — владелец (`$MHOWNER`), виртуальный канал, соединения и устройства ONLINE принадлежат ей; другая указывает на неё. У каждой записи своё рукопожатие, свои номера канала и свой буфер. Пакет TR принимается виртуальным каналом владельца, по какому бы открытому каналу узла он ни пришёл, а его номер канала сверяется с тем каналом, которым он пришёл. `mh$cc_chan` выбирает канал для пакета виртуального канала: из открытых, чей буфер его вмещает, — с наибольшим буфером, а пока прежний не хуже — прежний. Виртуальный канал закрывается вместе с последним каналом узла, не раньше; принятый SOLICIT_SRV закрывает остальные каналы узла: у загружающегося узла рабочих каналов уже нет. Владелец, чей виртуальный канал идёт через другой адаптер, не отдаётся новому узлу, когда таблица полна.

3.4 mh_tr.c

Транспорт виртуального канала: порядковые номера в длинных словах, накопительное подтверждение, окно не более чем в 16 пакетов, кэш пакетов, пришедших не по порядку, и повторная передача — 500 мс с удвоением до 3 с; 30 тайм-аутов подряд обрывают виртуальный канал. Причитающееся подтверждение уходит со следующим пакетом сервера либо отдельно, как велит `/ACK`. Окно заполняется сначала из очереди сообщений, затем порциями блочной передачи, стоящей во главе виртуального канала, которые изготовляются лишь тогда, когда есть место.

Каждый пакет с подтверждением несёт FLAG_EXT и LAST_ACCEPTED, равный подтверждению, как их посылает PEDRIVER. Узел VMS считает время от своего пакета до этого подтверждения задержкой канала (`SCACP SHOW CHANNEL`); без LAST_ACCEPTED он меряет канал только пробами.

У пакетов виртуального канала — 16 в окне, 64 в очереди, 32 в кэше приёма — буферы лежат в одном пуле, который выделяется вместе с каналом и рассчитан по MTU порта: 1426 байт данных на порцию в обычном Ethernet, 8926 с jumbo-кадрами при MTU 9000. Слот владеет своим буфером: сообщение из очереди копируется в слот окна, а слоты, освобождённые подтверждением, отдают свои буферы хвосту, который остаётся после сдвига окна.

Пакет уходит по каналу, который выбрал `mh$cc_chan`, с адресом и номером этого канала, а порция блочной передачи такого размера, какой берёт лучший открытый канал узла (`mh$cc_bufsz`). Если канал jumbo-кадров закрылся, а в окне порции по 8100 байт, их не берёт ни один открытый канал: другому адаптеру они не посылаются, виртуальный канал обрывается на следующем такте (`NOFIT`), и узел начинает его заново в размере тех каналов, что у него есть.

3.5 mh_ppd.c

Путь SCS к узлу по его виртуальному каналу: сервер отвечает на START узла сообщением STACK, берёт из START системный блок узла и открывает его транспорт. HOSTSHUT путь закрывает.

Загрузчик сателлита сам START не посылает никогда: он ждёт START своего сервера. По каналу с пометкой `boot` сервер посылает START каждую секунду, пока не придёт STACK (состояние ST_SENT), и отвечает на STACK сообщением ACK. START и STACK строит одна рутина, `s_mh$ppd_sb`, поэтому о сервере они говорят одно и то же: сателлит строит системный блок сервера своего диска из START, а его runtime SCS встречает тот же блок в STACK.

3.6 mh_scs.c

Принимающая половина SCS: сервер принимает соединения и никогда не просит их сам. Слушателей два: `SCS$DIRECTORY` и, когда обслуживаются диски, `MSCP$DISK`. Кредиты расходуются на каждое посланное прикладное сообщение и возвращаются со следующим. Каталог отвечает на запросы поиска: `MSCP$DISK` присутствует, когда обслуживаются диски, на прочие общеизвестные имена дан ответ «здесь отсутствует».

3.7 mh_blk.c

Блочные передачи. SEND_DATA (READ): порции уходят по мере открытия окна, по одной передаче за раз на виртуальный канал; конечное сообщение уходит лишь после того, как узел подтвердит последнюю порцию. REQUEST_DATA (WRITE, COMPARE): один запрос на все данные; порции приходят. Порция никогда не выходит за счётчик байтов своей команды; порция, пришедшая ни к чему из того, что ждёт сервер, отбрасывается, и о ней сообщается один раз (`BLKERR`).

3.8 mh_mscp.c

Сервер MSCP: устройства, хосты (по одному на соединение драйвера класса дисков), проверка команды, порядок команд на устройстве и сами команды. На немедленные команды (ABORT, GET COMMAND STATUS, GET UNIT STATUS, SET CONTROLLER CHARACTERISTICS и подобные) ответ даётся сразу; прочие становятся `MH$HRB`. Буферы данных берутся из пула в 16 МБ; команда, нашедшая его пустым, ждёт. Конечное сообщение, которое обрушило бы узел, задерживается (`MSCPERR`), а соединение завершается. Здесь же ведутся счётчики `UNITSTATS`, `BKSTATS` и `LATSTATS`.

3.9 mh_bk.c

Бэкенд. Сервер MSCP передаёт ему запрос (`MH$BKREQ`); тот уходит в io_uring одним шагом или несколькими. Устройство с секторами больше 512 октетов проводит запрос не на целые секторы через промежуточный буфер: READ читает целые секторы и копирует из них середину, WRITE читает первый и последний сектор (или находит их в кэше 128 последних записанных секторов), помещает свои данные между ними и пишет их все. Каждый WRITE такого устройства удерживает свой диапазон секторов; WRITE, встретивший удерживаемый диапазон, ждёт в порядке очереди. WRITE делается надёжным так, как велит `/SYNC`. ERASE пишет нули в файл размером до 1 МБ, а свыше пользуется `fallocate`. О запросе, пробывшем в бэкенде дольше 15 секунд, сообщается (`IOSTUCK`), и он отменяется.

Файл открывается дважды: один раз с `O_DIRECT` для ввода-вывода целыми блоками, другой раз через страничный кэш — для редкого счётчика, не кратного 512. Если файловая система отвергает `O_DIRECT`, файл остаётся на страничном кэше (`NODIRECT`).

3.10 mh_utl.c

Каталог сообщений (`s_msgtab`) и его регистрация, время OpenVMS и его часовой пояс, преобразования шестнадцатеричной записи и записи MAC-адреса.

3.11 mh_hlp.c

`/HELP`, на который отвечает библиотека справки через пакет HELP. Библиотека ищется в `MSCPHUB_HELPLIB`, в установленном дереве, рядом с образом и там, куда её положила сборка. Если сервер собран без пакета HELP, на `/HELP` он отвечает краткой сводкой.

3.12 mh_stat.c

Блок статистики, `/dev/shm/mscphub.`*узел*, режим 0640, создаётся при пуске сервера и удаляется при его остановке. `mh$stat_tick` на каждом такте прибавляет очередь каждого устройства к её сумме, а на каждом пятом такте копирует в блок состояние: SEQ делается нечётным, копия делается, SEQ делается чётным. `mh$stat_read` — рутина чтения и для MHMON, и для Modbus: она копирует блок между двумя чтениями SEQ и принимает копию, лишь если SEQ был чётным и не изменился; блок с другим MAGIC, VERSION или SIZE она отвергает. Блок, который не удалось создать, — это `NOSTAT`, предупреждение.

3.13 mh_mbus.c

Сервер Modbus/TCP: `mh$mbus_start` связывает сокет с адресом и начинает слушать в нити протокола, чтобы о неверном `/MODBUS` было сказано при пуске, а затем запускает нить, заблокировав все сигналы. Нить опрашивает слушающий сокет, не более 8 клиентов и EVENTFD, который её останавливает. `mh$mbus_answer` на каждый запрос берёт целую копию блока, раскладывает её по регистрам карты (руководство пользователя, приложение D) и отвечает на 03, 04 и 2B/0E; на всё прочее — исключение.

3.14 mhmon.c

MHMON — отдельный образ, собранный с той же библиотекой. Он отображает блок только для чтения, берёт копию каждые `/INTERVAL` на границах часов и вычисляет скорости по разностям двух копий за время, которое сервер на них проставил. Копия с другим PID или временем пуска начинает статистику заново; если PID исчез, отображение блока снимается, и MHMON отображает его снова, когда сервер возвращается. Экраны строятся через FAO в буфере и выводятся разом. Файл T4 пишется строка за строкой; его столбцы закрепляются при его начале, а узел или диск находится по своему имени.

3.15 mh_ods.c

Чтение тома Files-11 устройства, ODS-2 и ODS-5, только на чтение. `mh$ods_open` находит домашний блок (LBN 1 или первый блок, который называет сам себя) и заголовок INDEXF.SYS за битовой картой индексного файла; карта INDEXF.SYS даёт блок любого другого заголовка. `mh$ods_lookup` идёт по пути от MFD, каталог за каталогом, к старшей версии имени; псевдоним вроде SYSCOMMON в корне проходится как любая другая запись. `mh$ods_read` читает блоки файла по его карте, с заголовками расширения. Заголовок принимается, только если верна его контрольная сумма, уровень структуры 2 или 5 и FID тот, что просили, вместе с номером последовательности.

Том читается через дескриптор устройства, который может быть O_DIRECT: каждое чтение — целыми страницами в буфер читателя. Смещения 64-битные; `$MIN` из STARLET берёт `int` и для них не применяется. Блокировок читатель не берёт: он читает то, что узел не пишет, пока грузится сателлит.

3.16 mh_mop.c

MOP-сервер сателлитов из /SATELLITE*n*. У сателлита своё состояние (`MH$MOPSAT`): отвечают только названным адаптерам, каждая загрузка идёт своим ходом. Мультикастный REQUEST PROGRAM готовит загрузку — первую секцию образа `[`*root*`.SYSCOMMON.SYSEXE]APB.EXE`, прочитанную `mh_ods.c` за заголовком из HDRBLKCNT блоков, и параметры сателлита — и только потом уходит ASSISTANCE VOLUNTEER: сервер, который не может загрузить, загрузку и не предлагает. REQUEST PROGRAM серверу начинает загрузку с MEMORY LOAD 0. REQUEST MEMORY LOAD с номером посланного просит его снова, со следующим — двигает дальше; любой другой загрузку прекращает.

За образом идёт MEMORY LOAD с параметрами по адресу, где образ кончается: загрузчик ищет их сразу за собой. Это элементы из слова кода и слова длины: корень, имя сервера, имя службы устройства и его UNIT_ID, системный ID сервера, затем SCSNODE, SCSSYSTEMID, NISCS_PORT_SERV и NISCS_CONV_BOOT из `[`*root*`.SYSEXE]ALPHAVMSSYS.PAR` в том порядке, в каком они в этом файле, затем группа и пароль. Потом PARAMETER LOAD WITH TRANSFER ADDRESS. Загрузка от загрузочного сервера OpenVMS, записанная с сети, совпадает с этой октет в октет, кроме имени и системного ID сервера.

Глава 4. Структуры данных

Все они объявлены в `include/mh_def.h`.

Структура	Что это
`MH$CFG`	Настройки: значения квалификаторов
`MH$PORT`	Порт ЛВС и всё, что под ним: сокет, таблица узлов, счётчики
`MH$NODE`	Адаптер узла, замеченного в сети: его личность, канал к нему; владелец узла, виртуальный канал
`MH$VC`	Виртуальный канал к узлу: номера, окно, очереди, соединения
`MH$TRPKT`	Пакет в очереди передачи виртуального канала
`MH$CDT`	Соединение SCS: его состояние, кредиты, очередь
`MH$APPLMSG`	Прикладное сообщение, ждущее кредита
`MH$UNIT`	Устройство сервера MSCP: его имя, размер, состояние и счётчики
`MH$HRB`	Выполняемая команда
`MH$BKDEV`	Сторона устройства в бэкенде: дескрипторы, сектор, кэш записанных секторов
`MH$BKREQ`	Запрос к бэкенду
`MH$LAT`	Счётчики времени одной операции устройства
`MH$STAT`	Блок статистики: кто такой сервер, его счётчики, его узлы и устройства (`mh_stat.h`)
`MH$STNODE`, `MH$STUNIT`	Узел и устройство в блоке
`MH$MOPSAT`	Сателлит из /SATELLITE*n*: его адаптер, устройство и корень, состояние его загрузки
`MH$ODS`, `MH$ODSFILE`, `MH$ODSEXT`	Читаемый том Files-11, его файл, экстент файла

Владение устроено просто, потому что нить одна: структура принадлежит уровню, который объявляет её состояние, и нижний уровень никогда не освобождает того, что держит верхний. Выполняемая команда (`MH$HRB`) переживает своё соединение, если у диска ещё есть её запрос; она освобождается, когда бэкенд возвращает запрос.

`MH$STAT` — интерфейс между двумя образами и устроен как интерфейс: каждое поле несёт свой размер в типе, указателей нет, а `MH$K_STAT_VERSION` поднимается при любом изменении формата. MHMON отвергает блок другой версии (`STATVERS`).

Глава 5. Соглашения

Исходные тексты повсюду держатся стиля OpenVMS. Это не украшение: читатель, знающий соглашение, по одному виду имени определяет его время жизни и происхождение, а следующий программист читает код так, как его написал предыдущий. Правка, которая не следует этим правилам, не закончена.

5.1 Имена

Вид	Смысл
`l_name`	Локальная переменная
`s_name`	Статическое: переменная либо рутина, не экспортируемая из своего модуля
`a_name`	Параметр рутины
`g_name`	Глобальная переменная (в сервере таких нет)
`m_name`	Битовая маска
`v_name`	Битовое поле в структуре
`mh$module_name`	Экспортируемая рутина: подсистема, знак доллара, модуль, имя
`s_mh$module_name`	Статическая рутина
`MH$K_NAME`	Константа
`MH$_NAME`	Значение условия
`MH$NAME`	Тип структуры
`$NAME`	Макрос

Знак доллара разделяет слова имени, как в OpenVMS: `MH$K_BK_CACHE`, `$MHMSG`. Структура объявляется так, что тег несёт суффикс `_t`, а имя типа — знак доллара; скобка стоит в нулевой колонке, а члены сдвинуты отступом:

```
typedef struct mh_lat_t {
    uint64_t    n;          /* Commands                */
    uint64_t    sum, max;       /* From the command to the END          */
} MH$LAT;
```

Упорядоченные константы оформляются как `enum`. Целые несут свой размер в типе: `uint32_t`, `uint16_t`; значения в линии кладутся и берутся рутинами `nisca$put16`, `nisca$get32` и им подобными, никогда — приведением указателя.

5.2 Оформление

- **Whitesmiths.** Скобка блока стоит на отдельной строке на отступе того тела, которое она открывает. Скобки тела рутины стоят в нулевой колонке.
- **Только табуляция**, шаг восемь. Никаких пробелов для отступа.
- **Текст до 192-й колонки**, а не до 80-й.
- **Объявления в начале рутины**, без отступа, сгруппированы по типу.
- **Одиночный оператор под `if` скобок не получает.**
- **Составное условие заключается в скобки по частям**, чтобы ничто не зависело от старшинства операций: `if ( (a_len > MH$K_MAX) || !a_buf )`.
- **Счётчик цикла объявляется в `for`**: `for ( int i = 0; i < n; i++ )`.

Рутина оформляется так:

```
/*
**++
**  FUNCTIONAL DESCRIPTION:
**
**  Close the port.  The memberships go with the socket.
**
**  FORMAL PARAMETERS:
**
**  a_port      The port context
**
**  RETURN VALUE:
**  STS$K_SUCCESS.
**--
*/
int mh$lan_close    (
        MH$PORT *   a_port
            )
{
    if ( a_port->sd >= 0 )
        close(a_port->sd);

    a_port->sd = -1;

    return  STS$K_SUCCESS;
}
```

Блок над рутиной говорит, что она делает, каковы её `FORMAL PARAMETERS`, её `IMPLICIT INPUTS/OUTPUTS`, если они есть, и её `RETURN VALUE`. Комментарии подробны там, где код неочевиден, и отсутствуют там, где он очевиден: полэкрана над константой или над рутиной в одну строку — шум.

5.3 Шапка модуля

Каждый исходный файл начинается со своего опознания и своей истории:

```
#define __MODULE__  "MHLAN"
#define __IDENT__   "X01-10"
#define __REV__     "1.10.0"

/*
**++
**
**  FACILITY:   MSCPHUB - MSCP disk server for OpenVMS nodes, over NISCA
**
**  MODULE: mh_lan.c
**
**  ABSTRACT:   ...
**
**  DESCRIPTION: ...
**
**  AUTHOR: StarLet Squad and Ruslan R. Laishev (AKA: BadAss SysMan)
**
**  CREATION DATE:  17-OCT-2026
**
**  MODIFICATION HISTORY:
**
**      20-OCT-2026 RRL X01-10 : Initial public version.
**
**--
*/
```

Автор всегда «StarLet Squad and Ruslan R. Laishev (AKA: BadAss SysMan)», и каждая строка истории несёт `RRL` между датой и IDENT.

5.4 IDENT и REV

__IDENT__ — это версия на манер OpenVMS: X.*aa*-*bb*, а для небольшого исправления ошибки, не поднимающего *bb*, — X.*aa*-*bb*ECO*cc*; *cc* всегда из двух цифр. __REV__ — та же версия на манер Unix, выведенная из неё часть за частью: X.01-24 даёт 1.24.0, X.01-24ECO03 даёт 1.24.3. Модули, не менявшиеся с X01-24ECO2, сохраняют прежнюю форму без точки (X01-11ECO1 даёт 1.11.1); CMake принимает обе. Оба значения тщательно держатся вместе; CMake отказывается конфигурировать дерево, в котором __REV__ модуля не следует за его __IDENT__. Тег выпуска — сам IDENT продукта: X.01-24ECO03.

Модуль несёт IDENT последней версии, которая его изменила. IDENT продукта — это IDENT головного модуля, mscphub.c, и ниоткуда больше он не берётся.

5.5 Соглашение о вызовах

Рутина возвращает значение условия, а результат выдаёт через выходной параметр. Условие проверяется по младшему биту:

```
if ( !(1 & mh$lan_open(&s_port, $ASCPTR(&s_cfg.ifname))) )
    return  1;
```

Макрос `$MHMSG` возвращает переданное ему условие — благодаря этому сообщение и возврат укладываются в одно выражение.

5.6 Диагностика

- **То, что предназначено для чтения оператору**, идёт через `$MHMSG` — условие из каталога в `mh_utl.c` — в форме OpenVMS: `%MSCPHUB-F-IDCLASH, ...`. Текст находится в каталоге, а не в месте вызова.
- **То, что предназначено для чтения программисту**, идёт через `$LOG_FAO` с явно указанной серьёзностью `STS$K_*`.
- Ни одно сообщение, экран или файл сервера и MHMON не делается через `printf` ни в одном из его видов: всё идёт через движок FAO из StarLet, директивами OpenVMS: `!AZ`, `!UL`, `!SL`, `!UQ`, `!XW`, `!%S` и прочими. (Путь, собранный `snprintf`, — не вывод.) Тесты вольны писать как угодно.
- **Отказ рутины библиотеки времени исполнения Си сообщается там, где он произошёл**, с именем рутины и значением `errno`:

```
if ( 0 > (a_port->sd = socket(AF_PACKET, SOCK_RAW | SOCK_NONBLOCK | SOCK_CLOEXEC, htons(NISCA$K_ETHERTY
    return  $MHMSG(MH$_SYSERR, "socket(AF_PACKET)", errno, strerror(errno));
```

Вернуть условие наверх и оставить позади рассказ библиотеки времени исполнения — значит потерять единственное, что позволило бы этот отказ опознать.

5.7 Строки и длины

Строка носится вместе со своей длиной: счётные строки StarLet (`ASC`, `$ASCLEN`, `$ASCPTR`). Однажды известная длина повторно не вычисляется; дважды вызвать `strlen` над одной и той же строкой — ошибка стиля.

5.8 Файлы сборки и оболочка

- Никакого знака доллара в идентификаторе файла CMake: CMake принял бы его за начало ссылки. Константы сборки именуются `MH_`*NAME*.
- Никакого знака доллара в имени переменной окружения: `MSCPHUB_HELPLIB`, а не `MSCPHUB$HELPLIB`.

5.9 Документация

- Каждое руководство существует на английском и на русском языках, и оба говорят одно и то же.
- Английский текст прост: короткие предложения, употребительные слова. Многие читают его как второй язык.
- У каждого сообщения есть раздел в обеих библиотеках справки: что оно означает и что с ним делать.
- Руководства построены по образцу руководств OpenVMS: титульная страница с номером издания, предисловие, нумерованные главы, словарь команд, глава сообщений, глава поиска неисправностей.

Глава 6. Внесение изменений

6.1 Добавление квалификатора

1. Добавить поле в `MH$CFG` в `mh_def.h`.
2. Добавить строку в `s_opts` в `mscphub.c`: имя строчными буквами, поле, размер, тип.
3. Проверить его значение в `s_mh$check_cfg`; неверное значение — это `MH$_IVPARAM` с указанием того, что ожидается.
4. Описать его: раздел `## /NAME --` ... в обеих библиотеках справки, абзац на странице руководства (man), статья в словаре команд обоих руководств пользователя.

Тест `docs` не проходит, пока разделы и абзац не на месте: он берёт каждое имя из `s_opts` и ищет их.

6.2 Добавление сообщения

1. Добавить номер сообщения в `enum` в `mh_def.h` и условие `MH$_`*NAME* с его серьёзностью.
2. Добавить запись в `s_msgtab` в `mh_utl.c`: опознаватель, запятая, текст с директивами FAO.
3. Описать его: раздел *###NAME* в обеих библиотеках справки, статья в главе 6 обоих руководств пользователя.

Тест `docs` не проходит, пока разделы не на месте.

6.3 Добавление команды MSCP

Таблица команд, `s_opdef` в `mh_mscp.c`, даёт каждому коду операции длину принимаемой команды (0 — не принимается), длину её конечного сообщения, допустимые для неё модификаторы и её имя. Является ли команда немедленной, последовательной или выполняемой бок о бок, решается по её коду операции там, где команда принимается. Команда, которая перемещает данные, имеет шаги в `mh_mscp.c` и тип запроса в `mh_bk.c`. Каждое новое конечное сообщение обязано пройти проверку, задерживающую ответ, от которого узел потерпел бы крах; если у нового сообщения есть поля, которых проверка не знает, её следует расширить.

6.4 Добавление счётчика в MHMON и Modbus

1. Добавить поле в `MH$STAT`, `MH$STUNIT` или `MH$STNODE` в `mh_stat.h` и поднять `MH$K_STAT_VERSION`.
2. Копировать его в `mh$stat_tick` или в её вспомогательных рутинах в `mh_stat.c`.
3. Для MHMON: элемент или строка в `mhmon.c` и его столбец в файле T4.
4. Для Modbus: регистр в `s_mh$mb_layout` в `mh_mbus.c`, на незанятом месте карты; если регистр перемещается, поднять `MH$K_MB_MAPVERS`.
5. Описать его: руководство пользователя, раздел 4.8 или приложение D, на обоих языках.

6.5 Выпуск версии

1. Поднять `__IDENT__` и `__REV__` головного модуля и каждого изменённого модуля.
2. Добавить строку в историю изменений каждого изменённого файла.
3. Обновить титульные блоки руководств: номер издания, версию, дату и то, что издание заменяет.
4. `ctest` должен пройти.
5. `cmake --build build --target dist` изготавливает `MSCPHUB-`*ident*`.tar.gz` из зафиксированных исходных текстов.

Глава 7. Система сборки

7.1 Состав

Каталог	Содержимое
`src/`	Модули
`include/`	`mh_def.h`, `nisca.h`, `mscp.h`
`doc/`	Руководства, исходные тексты библиотек справки, страница руководства (man), `mkpdf.py`
`etc/`	Образец файла параметров, юниты systemd, скрипты OpenRC и System V init
`tests/`	Тесты
`cmake/`	Сценарий удаления

7.2 Зависимости

Пакет	Назначение
StarLet 1.6 или новее	Сообщения, протоколирование, командная строка, счётные строки, FAO
liburing	Бэкенд
HELP (необязательно)	Библиотека справки и `/HELP`
pandoc и XeLaTeX (необязательно)	PDF руководств

7.3 Ключи

Ключ	По умолчанию	Действие
`CMAKE_INSTALL_PREFIX`	`/opt/mscphub`	Куда устанавливается сервер
`MH_STRICT`	OFF	Предупреждения компилятора становятся ошибками
`BUILD_TESTING`	ON	Тесты собираются
`MH_SYSTEMD`	ON там, где работает systemd	Устанавливаются юниты systemd (но никогда не включаются)
`MH_CONFDIR`	`/etc/mscphub` (`etc/mscphub` другого префикса)	Где лежит файл параметров службы
`MH_INIT`	`OPENRC`, `SYSV` или `NONE`, по компьютеру	Скрипты init ставятся в `MH_INITDIR`, `/etc/init.d` (но никогда не включаются)
`MH_PDF`	OFF	Руководства выводятся в PDF

7.4 Проверки при конфигурировании

CMake отвергает дерево, в котором `__REV__` модуля не следует за его `__IDENT__` (раздел 5.4). Версия продукта берётся из `src/mscphub.c`.

7.5 PDF руководств

При `-DMH_PDF=ON` сценарий `doc/mkpdf.py` выводит каждое руководство через pandoc и XeLaTeX в PDF рядом с его исходным текстом. Выведенные руководства хранятся в репозитории вместе с исходными текстами, чтобы читателю не был нужен TeX для их чтения.

7.6 Дистрибутив

`cmake --build build --target dist` изготавливает `MSCPHUB-`*ident*`.tar.gz` командой `git archive HEAD`: зафиксированные исходные тексты, руководства и тесты — и ничего из дерева сборки. Чего нет в репозитории, того нет и в дистрибутиве.

Глава 8. Тесты

8.1 Что есть

Тест	Что он стережёт
`wire`	Кадры, которые строит сервер, октет за октетом, против эталонных кадров OpenVMS V8.3 (`tests/t_frames.h`); рукопожатие канала
`vc`	Виртуальный канал и путь: повторная передача, дубликаты, нарушенный порядок, старый канал, 30 тайм-аутов, HOSTSHUT, перезапуск
`scs`	SCS и каталог; сервер MSCP: STCON, GET UNIT STATUS, ONLINE, AVAILABLE, проверки команды, кредиты
`blk`	READ, WRITE, COMPARE и ERASE через блочные передачи; порядок команд; ABORT; соединение, уходящее, пока у диска есть команда
`bk`	Бэкенд: io_uring, чтение-изменение-запись, диапазоны секторов, кэш записанных секторов, ERASE, групповая фиксация, прямой ввод-вывод
`mchan`	Узел с двумя адаптерами: второй канал обслуживает виртуальный канал первого, пакет уходит каналом с бóльшим буфером и с его номером, виртуальный канал переживает канал, пакет, слишком большой для оставшихся каналов, сразу его обрывает, SOLICIT_SRV закрывает прочие каналы
`ods`	Чтение Files-11 на маленьком томе, который делает тест (`tests/t_vol.h`): домашний блок, INDEXF.SYS, псевдоним, версии, экстенты и заголовки расширения, неверный номер последовательности или контрольная сумма
`mop`	MOP-сервер: сателлит загружается с тома из `ods`, V4 и V3, параметры поэлементно, таймеры, запросы без ответа

Тест	Что он стережёт
`stat`	Блок статистики: создаётся, копируется каждые 100 мс, никогда не даёт разорванной копии при записи из другой нити, блок другого формата отвергается, очереди; ответы Modbus
`t4`	MHMON записывает опубликованный блок: заголовок T4, одни и те же поля в каждой строке, скорость устройства
`init`	Скрипты System V: без файла параметров не запускается, сборщик T4 запускается, показывает состояние, останавливается и закрывает свой файл
`params`	Проверки квалификаторов
`docs`	Каждый квалификатор сервера и MHMON и каждое сообщение описаны
`helplib`	`/HELP` отвечает из библиотеки (с пакетом HELP)

8.2 Правила

- Правка, меняющая то, что посылает сервер, не закончена без теста, который до неё не проходит, а после неё проходит.
- Тест проверяется так: ломается код, который он стережёт, и надо увидеть, что тест не проходит.
- `ctest` проходит до того, как что-либо устанавливается.

Приложение A. Известные ограничения

Ограничение	Где оно живёт
8 дисков на сервер	От `/UNIT0` до `/UNIT7` в `s_opts`, `mscphub.c`
Один сетевой адаптер	`mh_lan.c`
8 клиентов Modbus одновременно	`mh_mbus.c`
64 запоминаемых узла	`mh_cc.c`, `MANYNODES`
Окно в 16 пакетов на виртуальный канал	`mh_tr.c`
Jumbo-кадры до MTU 9000	`MH$K_LAN_MAXMTU`, `mh_def.h`
Нет диска-члена теневого набора (нет forced error)	`mh_mscp.c`
Нет лент	не реализовано
8 сателлитов, загружаемых по MOP	От `/SATELLITE0` до `/SATELLITE7` в `s_opts`, `mscphub.c`
MOP только для сателлитов Alpha; нет BOOTP и TFTP: сателлит Integrity загружает узел OpenVMS	не реализовано
Нет кластера поверх IP	не реализовано